

2. Data Structures

Contents

Mo struct	2
Mo with updates	2
2D Prefix Sum	4
Multiset Lazy deletion	4
BIT, Fenwick Tree	5
BIT Range / Fenwick Range	5
2D BIT / 2D fenwick	6
Segment Tree	7
Segment Tree (Recursive)	7
2D Segment Tree	8
Lazy Segment Tree	9
Dynamic Lazy Segment Tree	11
Segment Tree Beats	12
Dynamic Persistent Segment Tree	15
Wavelet Tree	16
Implicit Treap	17
Sparse Table	19
2D Sparse Table	20
DSU	21
Persistent DSU	21
Bipartite DSU	22
Monotonic Stack / Queue	23
Ordered Data Structures (pb_ds)	24
BucketList	24
Full Dynamic Array	25

Mo struct

```

1  const int N = 1e5 + 5, sq = 317;
2  struct query{
3      int l, r, i;
4      bool operator<(const query &other) const {
5          if (l / sq != other.l / sq)
6              return l / sq < other.l / sq;
7          return (l / sq & 1 ? r < other.r : other.r < r);
8      }
9  };

```

Mo with updates

```

1  const int N = 2e5 + 5;
2  int a[N];
3  vector<int> coords;
4  long long curAns, ans[N];
5
6  struct Query {
7      int l, r, t, id, blk_l, blk_r;
8
9      bool operator<(const Query& o) const {
10         if (blk_l != o.blk_l) return blk_l < o.blk_l;
11         if (blk_r != o.blk_r) return (blk_l & 1) ? blk_r < o.blk_r : blk_r > o.blk_r;
12         return (blk_r & 1) ? t < o.t : t > o.t;
13     }
14 };
15
16 struct Update {
17     int p, v;
18 };
19
20 vector<Query> queries;
21 vector<Update> updates;
22 int vals[N];
23
24 inline void add(int i) {
25     if (++vals[a[i]] == 1) curAns += coords[a[i]];
26 }
27
28 inline void del(int i) {
29     if (!--vals[a[i]]) curAns -= coords[a[i]];
30 }
31
32 inline void do_update(int t, int l, int r) {
33     auto &u = updates[t];
34     if (l <= u.p && u.p <= r) del(u.p);
35     swap(a[u.p], u.v);
36     if (l <= u.p && u.p <= r) add(u.p);
37 }
38
39 void process(int n) {

```

```

40     constexpr int block = 700;
41     for (auto &q: queries) {
42         q.blk_l = q.l / block;
43         q.blk_r = q.r / block;
44     }
45     sort(queries.begin(), queries.end());
46
47     int l = 1, r = 0, t = 0;
48     for (const auto &q: queries) {
49         while (t < q.t) do_update(t++, l, r);
50         while (t > q.t) do_update(--t, l, r);
51         while (l > q.l) add(--l);
52         while (r < q.r) add(++r);
53         while (l < q.l) del(l++);
54         while (r > q.r) del(r--);
55
56         ans[q.id] = curAns;
57     }
58 }
59
60 void solve() {
61     int n;
62     cin >> n;
63     coords.reserve(n << 1);
64     for (int i = 0; i < n; i++) cin >> a[i], coords.push_back(a[i]);
65     int q, j = 0;
66     cin >> q;
67     for (int i = 0; i < q; i++) {
68         string s;
69         cin >> s;
70         if (s.front() == 'Q') {
71             int l, r;
72             cin >> l >> r;
73             queries.push_back({--l, --r, (int) updates.size(), j, 0, 0});
74             j++;
75         } else {
76             int idx, val;
77             cin >> idx >> val;
78             --idx;
79             updates.push_back({idx, val});
80             coords.push_back(val);
81         }
82     }
83
84     sort(coords.begin(), coords.end());
85     coords.erase(unique(coords.begin(), coords.end()), coords.end());
86     for (int i = 0; i < n; i++)
87         a[i] = lower_bound(coords.begin(), coords.end(), a[i]) - coords.begin();
88     for (auto &o: updates)
89         o.v = lower_bound(coords.begin(), coords.end(), o.v) - coords.begin();
90
91     // pass n to the process function
92     process(n);
93
94     for (int i = 0; i < j; i++) cout << ans[i] << ' ';
95
96     queries.clear();

```

```

97     updates.clear();
98     coords.clear();
99     curAns = 0;
100 }

```

2D Prefix Sum

```

1 // construction
2 for (int i = 1; i <= N; i++) {
3     for (int j = 1; j <= N; j++) {
4         prefix[i][j] =
5             arr[i][j]
6             + prefix[i - 1][j]
7             + prefix[i][j - 1]
8             - prefix[i - 1][j - 1];
9     }
10 }
11
12 // query
13 pfx[to_row][to_col]
14 - pfx[from_row - 1][to_col]
15 - pfx[to_row][from_col - 1]
16 + pfx[from_row - 1][from_col - 1];
17
18 // partial: add value v to subrectangle [from_row, from_col] to [to_row, to_col]
19 diff[from_row][from_col] += v;
20 diff[from_row][to_col + 1] -= v;
21 diff[to_row + 1][from_col] -= v;
22 diff[to_row + 1][to_col + 1] += v;
23 // then construct diff, grid[i][j] += diff[i][j];

```

Multiset Lazy deletion

```

1 template<typename T, typename Compare = less<T>>
2 struct MS {
3     priority_queue<T, vector<T>, Compare> pq, del;
4     void normalize(){
5         while(!pq.empty() && !del.empty() && pq.top() == del.top()){
6             pq.pop();
7             del.pop();
8         }
9     }
10    bool empty() { normalize(); return size() == 0; }
11    int size() { return (int)pq.size() - (int)del.size(); }
12    void insert(T x) { pq.push(x); }
13    void erase(T x) { del.push(x); }
14    T top() { normalize(); return pq.top(); }
15    void pop() { normalize(); pq.pop(); }
16    void clear() {
17        while(!pq.empty()) pq.pop();
18        while(!del.empty()) del.pop();
19    }
20 };

```

BIT, Fenwick Tree

```
1 struct BIT { // 0-based
2     int n;
3     vector<ll> tree;
4
5     BIT(int size) : n(size + 2), tree(n + 1) {}
6     void add(int i, ll val) {
7         for(i++; i <= n; i += i & -i) tree[i] += val;
8     }
9     ll query(int i) {
10        ll sum = 0;
11        for (i++; i > 0; i &= i - 1) sum += tree[i];
12        return sum;
13    }
14    int lower_bound(ll target) {
15        int i = 0;
16        ll curr = 0;
17        for (int mask = 1 << __lg(n); mask > 0; mask >>= 1) {
18            if (i + mask <= n && curr + tree[i + mask] < target) {
19                curr += tree[i += mask];
20            }
21        }
22        return i;
23    }
24 };
```

BIT Range / Fenwick Range

```
1 template<typename T>
2 class BitR { // 0-based
3     int n;
4     vector<T> f, s;
5     void add(vector<T> &a, int i, T val) {
6         for(; i < n; i += i & -i)
7             a[i] += val;
8     }
9 public:
10    BitR(int n) : n(n + 5), f(n + 6), s(n + 6) { }
11
12    void add(int i, T val) { add(s, i + 1, -val); }
13
14    T query_point(int i) {
15        if (!i) return query(0);
16        return query(i) - query(i - 1);
17    }
18    void set(int i, T val) {
19        int c = query_point(i);
20        add(i, val - c);
21    }
22
23    void add(int l, int r, T val) {
24        l++, r++;
25        add(f, l, val);
26        add(f, r + 1, -val);
27    }
```

```

27         add(s, l, val * (l - 1));
28         add(s, r + 1, -val * r);
29     }
30
31     T query(int ii) {
32         ii++;
33         T sum = 0;
34         int i = ii;
35         for(; i > 0; i ^= i & -i)
36             sum += f[i];
37         sum *= ii;
38         i = ii;
39         for(; i > 0; i ^= i & -i)
40             sum -= s[i];
41         return sum;
42     }
43
44     T range(int l, int r) { return query(r) - query(l - 1); }
45 };

```

2D BIT / 2D fenwick

```

1  template<typename T>
2  struct BIT2D { // 1-based
3      int n, m;
4      vector<vector<T>>> tree;
5
6      BIT2D(int n, int m) : n(n), m(m), tree(n + 2, vector<T>(m + 2, 0)) {}
7
8      void update(int x, int y, T val) {
9          for (int i = x; i <= n; i += i & -i) {
10             for (int j = y; j <= m; j += j & -j) {
11                 tree[i][j] += val;
12             }
13         }
14     }
15
16     T getPrefix(int x, int y) {
17         if (x <= 0 || y <= 0) return 0;
18         T ret = 0; // change default value
19         for (int i = x; i > 0; i &= i - 1) {
20             for (int j = y; j > 0; j &= j - 1) {
21                 ret += tree[i][j];
22             }
23         }
24         return ret;
25     }
26
27     T getSquare(int xl, int yl, int xr, int yr) { // change operation
28         return getPrefix(xr, yr) + getPrefix(xl - 1, yl - 1) -
29             getPrefix(xr, yl - 1) - getPrefix(xl - 1, yr);
30     }
31 };

```

Segment Tree

```
1 struct info {
2     long long sum = 0;
3     info(long long sum = 0) : sum(sum) {}
4     friend info operator+(const info &l, const info &r) {
5         return {l.sum + r.sum};
6     }
7 };
8
9 template<class info>
10 struct segmentTreeIterative {
11     int n;
12     vector<info> tree;
13
14     explicit segmentTreeIterative(int n) : n(n), tree(n << 1) {}
15
16     template<class U>
17     explicit segmentTreeIterative(const vector<U> &arr) : n(arr.size()),
18         tree(n << 1) {
19         for(int i = 0; i < n; i++) tree[i + n] = info(arr[i]);
20         for(int i = n - 1; i > 0; i--) tree[i] = tree[i << 1] + tree[i << 1 | 1];
21     }
22
23     void set(int i, info v) {
24         for(tree[i += n] = v; i >>= 1; )
25             tree[i] = tree[i << 1] + tree[i << 1 | 1];
26     }
27
28     info get(int l, int r) {
29         info resL, resR;
30         for(l += n, r += n + 1; l < r; l >>= 1, r >>= 1) {
31             if(l & 1) resL = resL + tree[l++];
32             if(r & 1) resR = tree[--r] + resR;
33         }
34         return resL + resR;
35     }
36 };
```

Segment Tree (Recursive)

```
1 struct info {
2     long long sum = 0;
3     info(long long sum = 0) : sum(sum) {}
4     friend info operator+(const info &l, const info &r) {
5         return {l.sum + r.sum};
6     }
7 };
8
9 template<class info>
10 struct dynamicSegmentTree {
11     struct node { int l = 0, r = 0; info v; };
12     vector<node> tr;
13     int n, root = 0;
14
15     explicit dynamicSegmentTree(int n = 1e9, int expectedOps = 1) : n(n), tr(1) {
```

```

16         tr.reserve(expectedOps * __lg(n) * 2);
17     }
18
19     info get(int x, int lx, int rx, int l, int r) {
20         if (!x || lx > r || l > rx) return info();
21         if (lx >= l && rx <= r) return tr[x].v;
22         int m = (lx + rx) >> 1;
23         return get(tr[x].l, lx, m, l, r) + get(tr[x].r, m + 1, rx, l, r);
24     }
25
26     int set(int x, int lx, int rx, int i, info val) {
27         if (i < lx || i > rx) return x;
28         if (!x) x = tr.size(), tr.emplace_back();
29         if (lx == rx) return tr[x].v = val, x;
30         int m = (lx + rx) >> 1;
31         tr[x].l = set(tr[x].l, lx, m, i, val);
32         tr[x].r = set(tr[x].r, m + 1, rx, i, val);
33         return tr[x].v = tr[tr[x].l].v + tr[tr[x].r].v, x;
34     }
35
36     void set(int i, info v) { root = set(root, 0, n, i, v); }
37     info get(int l, int r) { return get(root, 0, n, l, r); }
38 };

```

2D Segment Tree

```

1  struct info {
2      long long sum = 0;
3      info(long long sum = 0) : sum(sum) {}
4      friend info operator+(const info &l, const info &r) {
5          return {l.sum + r.sum};
6      }
7  };
8
9  template<class info>
10 struct segmentTree2d {
11     int nx, ny;
12     vector<vector<info>> tree;
13
14     explicit segmentTree2d(int n, int m) : nx(n), ny(m), tree(nx << 1,
15         vector<info>(ny << 1)) {}
16
17     template<class U>
18     explicit segmentTree2d(const vector<vector<U>> &a) {
19         nx = a.size();
20         ny = nx ? a[0].size() : 0;
21         tree.assign(nx << 1, vector<info>(ny << 1));
22
23         for(int i = 0; i < nx; i++) {
24             for(int j = 0; j < ny; j++) tree[i + nx][j + ny] = info(a[i][j]);
25             for(int y = ny - 1; y > 0; y--)
26                 tree[i + nx][y] = tree[i + nx][y << 1] + tree[i + nx][y << 1 | 1];
27         }
28         for(int x = nx - 1; x > 0; x--)
29             for(int y = 0; y < (ny << 1); y++)

```



```

30         tree[x][y] = tree[x << 1][y] + tree[x << 1 | 1][y];
31     }
32
33     void set(int i, int j, info v) {
34         if (i < 0 || i >= nx || j < 0 || j >= ny) return;
35         int x = i + nx, y = j + ny;
36         tree[x][y] = v;
37         for(int yy = y >> 1; yy > 0; yy >>= 1)
38             tree[x][yy] = tree[x][yy << 1] + tree[x][yy << 1 | 1];
39
40         for(int xx = x >> 1; xx > 0; xx >>= 1) {
41             tree[xx][y] = tree[xx << 1][y] + tree[xx << 1 | 1][y];
42             for(int yy = y >> 1; yy > 0; yy >>= 1)
43                 tree[xx][yy] = tree[xx][yy << 1] + tree[xx][yy << 1 | 1];
44         }
45     }
46
47     info queryY(int nodeX, int y1, int y2) const {
48         info resL, resR;
49         for(int l = y1 + ny, r = y2 + ny + 1; l < r; l >>= 1, r >>= 1) {
50             if (l & 1) resL = resL + tree[nodeX][l++];
51             if (r & 1) resR = tree[nodeX][--r] + resR;
52         }
53         return resL + resR;
54     }
55
56     info get(int x1, int y1, int x2, int y2) {
57         if (!nx || !ny) return info();
58         x1 = max(x1, 0); y1 = max(y1, 0);
59         x2 = min(x2, nx - 1); y2 = min(y2, ny - 1);
60         if (x1 > x2 || y1 > y2) return info();
61
62         info resL, resR;
63         for(int l = x1 + nx, r = x2 + nx + 1; l < r; l >>= 1, r >>= 1) {
64             if (l & 1) resL = resL + queryY(l++, y1, y2);
65             if (r & 1) resR = queryY(--r, y1, y2) + resR;
66         }
67         return resL + resR;
68     }
69 };

```

Lazy Segment Tree

```

1 struct tag {
2     ll add = 0;
3     void apply(const tag &p) {
4         add += p.add;
5     }
6 };
7
8 struct info {
9     ll sum = 0;
10    info(ll sum = 0) : sum(sum) {}
11    void apply(const tag &lz, int lx, int rx) {
12        sum += 1ll * (rx - lx + 1) * lz.add;
13    }

```

```

14     friend info operator+(const info &l, const info &r) {
15         return {l.sum + r.sum};
16     }
17 };
18
19 template<class info, class tag>
20 struct lazySegment {
21     int n;
22     vector<info> tr;
23     vector<tag> lz;
24
25     explicit lazySegment(int _n) : n(_n + 1), tr(4 << __lg(n)), lz(4 << __lg(n)) {}
26     template<class U>
27     explicit lazySegment(const U &arr) : n(arr.size()), tr(4 << __lg(n)),
28         lz(4 << __lg(n)) {
29         build(1, 0, n - 1, arr);
30     }
31
32     void push(int x, int lx, int rx) {
33         if (lx != rx) lz[x << 1].apply(lz[x]), lz[x << 1 | 1].apply(lz[x]);
34         tr[x].apply(lz[x], lx, rx);
35         lz[x] = tag();
36     }
37
38     template<class U>
39     void build(int x, int lx, int rx, const U &arr) {
40         if (lx == rx) return void(tr[x] = arr[lx]);
41         int m = (lx + rx) >> 1;
42         build(x << 1, lx, m, arr), build(x << 1 | 1, m + 1, rx, arr);
43         tr[x] = tr[x << 1] + tr[x << 1 | 1];
44     }
45
46     info get(int x, int lx, int rx, int l, int r) {
47         push(x, lx, rx);
48         if (lx > r || l > rx) return info();
49         if (lx >= l && rx <= r) return tr[x];
50         int m = (lx + rx) >> 1;
51         return get(x << 1, lx, m, l, r) + get(x << 1 | 1, m + 1, rx, l, r);
52     }
53
54     void set(int x, int lx, int rx, int i, info val) {
55         push(x, lx, rx);
56         if (i < lx || i > rx) return;
57         if (lx == rx) return void(tr[x] = val);
58         int m = (lx + rx) >> 1;
59         set(x << 1, lx, m, i, val), set(x << 1 | 1, m + 1, rx, i, val);
60         tr[x] = tr[x << 1] + tr[x << 1 | 1];
61     }
62
63     void applyRange(int x, int lx, int rx, int l, int r, tag v) {
64         push(x, lx, rx);
65         if (lx > r || l > rx) return;
66         if (lx >= l && rx <= r) return lz[x] = v, push(x, lx, rx);
67         int m = (lx + rx) >> 1;
68         applyRange(x << 1, lx, m, l, r, v),
69         applyRange(x << 1 | 1, m + 1, rx, l, r, v);

```

```

70     tr[x] = tr[x << 1] + tr[x << 1 | 1];
71 }
72
73 void set(int i, info v) { set(1, 0, n - 1, i, v); }
74 void applyRange(int l, int r, tag v) { applyRange(1, 0, n - 1, l, r, v); }
75 info get(int l, int r) { return get(1, 0, n - 1, l, r); }
76 };

```

Dynamic Lazy Segment Tree

```

1  template<class info, class tag>
2  struct dynamicLazySegmentTree {
3      struct node { int l = 0, r = 0; info v; tag t; };
4      vector<node> tr;
5      int n, root = 0;
6
7      explicit dynamicLazySegmentTree(int n = 1e9) : n(n), tr(1) {}
8
9      void push(int x, int lx, int rx) {
10         if (lx == rx) return void(tr[x].t = tag());
11         if (!tr[x].l) tr[x].l = tr.size(), tr.emplace_back();
12         if (!tr[x].r) tr[x].r = tr.size(), tr.emplace_back();
13
14         int m = (lx + rx) >> 1, l = tr[x].l, r = tr[x].r;
15         tr[l].t.apply(tr[x].t), tr[l].v.apply(tr[x].t, lx, m);
16         tr[r].t.apply(tr[x].t), tr[r].v.apply(tr[x].t, m + 1, rx);
17         tr[x].t = tag();
18     }
19
20     int set(int x, int lx, int rx, int i, info val) {
21         if (i < lx || i > rx) return x;
22         if (!x) x = tr.size(), tr.emplace_back();
23         if (lx == rx) {
24             tr[x].v = val, tr[x].t = tag();
25             return x;
26         }
27         push(x, lx, rx);
28         int m = (lx + rx) >> 1;
29         tr[x].l = set(tr[x].l, lx, m, i, val);
30         tr[x].r = set(tr[x].r, m + 1, rx, i, val);
31         return tr[x].v = tr[tr[x].l].v + tr[tr[x].r].v, x;
32     }
33
34     int applyRange(int x, int lx, int rx, int l, int r, tag val) {
35         if (lx > r || l > rx) return x;
36         if (!x) x = tr.size(), tr.emplace_back();
37         if (lx >= l && rx <= r) {
38             tr[x].t.apply(val);
39             tr[x].v.apply(val, lx, rx);
40             return x;
41         }
42         push(x, lx, rx);
43         int m = (lx + rx) >> 1;
44         tr[x].l = applyRange(tr[x].l, lx, m, l, r, val);
45         tr[x].r = applyRange(tr[x].r, m + 1, rx, l, r, val);

```

```

46     return tr[x].v = tr[tr[x].l].v + tr[tr[x].r].v, x;
47 }
48
49 info get(int x, int lx, int rx, int l, int r) {
50     if (!x || lx > r || l > rx) return info();
51     if (lx >= l && rx <= r) return tr[x].v;
52     push(x, lx, rx);
53     int m = (lx + rx) >> 1;
54     return get(tr[x].l, lx, m, l, r) + get(tr[x].r, m + 1, rx, l, r);
55 }
56
57 void set(int i, info v) { root = set(root, 0, n, i, v); }
58 void applyRange(int l, int r, tag v) { root = applyRange(root, 0, n, l, r, v); }
59 info get(int l, int r) { return get(root, 0, n, l, r); }
60 };

```

Segment Tree Beats

```

1  struct segtreebeats {
2      static const ll INF = 2e18, UNSET = LLONG_MIN;
3
4      struct Node {
5          ll sum, mx1, mx2, mxc, mn1, mn2, mnc, d_gcd, lz_add, lz_set;
6      };
7
8      int sz;
9      vector<Node> tree;
10
11      Node leaf(ll v) { return {v, v, -INF, 1, v, INF, 1, 0, 0, UNSET}; }
12
13      segtreebeats(int n, const vector<ll> &a) {
14          for (sz = 1; sz < n; sz <= 1);
15          tree.assign(sz << 1, leaf(0));
16          build(a, n, 0, 0, sz - 1);
17      }
18
19      Node merge(const Node &L, const Node &R) {
20          Node U = {L.sum + R.sum, 0, 0, 0, 0, 0, 0, gcd(L.d_gcd, R.d_gcd), 0, UNSET};
21
22          if (L.mx1 == R.mx1) U.mx1 = L.mx1, U.mx2 = max(L.mx2, R.mx2), U.mxc = L.mxc + R.mxc;
23          else if (L.mx1 > R.mx1) U.mx1 = L.mx1, U.mx2 = max(L.mx2, R.mx1), U.mxc = L.mxc;
24          else U.mx1 = R.mx1, U.mx2 = max(L.mx1, R.mx2), U.mxc = R.mxc;
25
26          if (L.mn1 == R.mn1) U.mn1 = L.mn1, U.mn2 = min(L.mn2, R.mn2), U.mnc = L.mnc + R.mnc;
27          else if (L.mn1 < R.mn1) U.mn1 = L.mn1, U.mn2 = min(L.mn2, R.mn1), U.mnc = L.mnc;
28          else U.mn1 = R.mn1, U.mn2 = min(L.mn1, R.mn2), U.mnc = R.mnc;
29
30          ll aL = L.mx2, aR = R.mx2;
31          if (aL != -INF && aL != L.mn1 && aR != -INF && aR != R.mn1)
32              U.d_gcd = gcd(U.d_gcd, abs(aL - aR));
33
34          ll any = UNSET;
35          if (aL != -INF && aL != L.mn1) any = aL;
36          else if (aR != -INF && aR != R.mn1) any = aR;
37

```

```

38     for (ll val : {L.mn1, L.mx1, R.mn1, R.mx1}) {
39         if (val != U.mn1 && val != U.mx1) {
40             if (any != UNSET) U.d_gcd = gcd(U.d_gcd, abs(val - any));
41             else any = val;
42         }
43     }
44     return U;
45 }
46
47 void apply_set(int x, int lx, int rx, ll v) {
48     ll len = rx - lx + 1;
49     tree[x] = {len * v, v, -INF, len, v, INF, len, 0, 0, v};
50 }
51
52 void apply_add(int x, int lx, int rx, ll v) {
53     if (!v) return;
54     auto &nd = tree[x];
55     if (nd.lz_set != UNSET) return apply_set(x, lx, rx, nd.lz_set + v);
56     if (nd.mx1 == nd.mn1) return apply_set(x, lx, rx, nd.mn1 + v);
57     ll len = rx - lx + 1;
58     nd.sum += len * v;
59     nd.mx1 += v; if (nd.mx2 != -INF) nd.mx2 += v;
60     nd.mn1 += v; if (nd.mn2 != INF) nd.mn2 += v;
61     nd.lz_add += v;
62 }
63
64 void apply_chmin(int x, int lx, int rx, ll v) {
65     auto &nd = tree[x];
66     if (nd.mx1 <= v) return;
67     if (nd.mn1 >= v) return apply_set(x, lx, rx, v);
68     if (nd.mn2 == nd.mx1) nd.mn2 = v;
69     nd.sum -= (nd.mx1 - v) * nd.mxc;
70     nd.mx1 = v;
71 }
72
73 void apply_chmax(int x, int lx, int rx, ll v) {
74     auto &nd = tree[x];
75     if (nd.mn1 >= v) return;
76     if (nd.mx1 <= v) return apply_set(x, lx, rx, v);
77     if (nd.mx2 == nd.mn1) nd.mx2 = v;
78     nd.sum += (v - nd.mn1) * nd.mnc;
79     nd.mn1 = v;
80 }
81
82 void push(int x, int lx, int rx) {
83     if (lx == rx) return tree[x].lz_add = 0, tree[x].lz_set = UNSET, void();
84     int m = (lx + rx) >> 1, lc = x * 2 + 1, rc = x * 2 + 2;
85
86     if (tree[x].lz_set != UNSET) {
87         apply_set(lc, lx, m, tree[x].lz_set);
88         apply_set(rc, m + 1, rx, tree[x].lz_set);
89         tree[x].lz_set = UNSET;
90     }
91     if (tree[x].lz_add) {
92         apply_add(lc, lx, m, tree[x].lz_add);
93         apply_add(rc, m + 1, rx, tree[x].lz_add);

```

```

94         tree[x].lz_add = 0;
95     }
96     if (tree[lc].mx1 > tree[x].mx1) apply_chmin(lc, lx, m, tree[x].mx1);
97     if (tree[rc].mx1 > tree[x].mx1) apply_chmin(rc, m + 1, rx, tree[x].mx1);
98     if (tree[lc].mn1 < tree[x].mn1) apply_chmax(lc, lx, m, tree[x].mn1);
99     if (tree[rc].mn1 < tree[x].mn1) apply_chmax(rc, m + 1, rx, tree[x].mn1);
100 }
101
102 void build(const vector<ll> &a, int n, int x, int lx, int rx) {
103     if (lx == rx) return void(tree[x] = (lx < n) ? leaf(a[lx]) : leaf(0));
104     int m = (lx + rx) >> 1;
105     build(a, n, x * 2 + 1, lx, m);
106     build(a, n, x * 2 + 2, m + 1, rx);
107     tree[x] = merge(tree[x * 2 + 1], tree[x * 2 + 2]);
108 }
109
110 void chmin(int l, int r, ll v, int x, int lx, int rx) {
111     if (lx > r || rx < l || tree[x].mx1 <= v) return;
112     if (lx >= l && rx <= r && tree[x].mx2 < v) return apply_chmin(x, lx, rx, v);
113     push(x, lx, rx);
114     int m = (lx + rx) >> 1;
115     chmin(l, r, v, x * 2 + 1, lx, m);
116     chmin(l, r, v, x * 2 + 2, m + 1, rx);
117     tree[x] = merge(tree[x * 2 + 1], tree[x * 2 + 2]);
118 }
119
120 void chmax(int l, int r, ll v, int x, int lx, int rx) {
121     if (lx > r || rx < l || tree[x].mn1 >= v) return;
122     if (lx >= l && rx <= r && tree[x].mn2 > v) return apply_chmax(x, lx, rx, v);
123     push(x, lx, rx);
124     int m = (lx + rx) >> 1;
125     chmax(l, r, v, x * 2 + 1, lx, m);
126     chmax(l, r, v, x * 2 + 2, m + 1, rx);
127     tree[x] = merge(tree[x * 2 + 1], tree[x * 2 + 2]);
128 }
129
130 void assign(int l, int r, ll v, int x, int lx, int rx) {
131     if (lx > r || rx < l) return;
132     if (lx >= l && rx <= r) return apply_set(x, lx, rx, v);
133     push(x, lx, rx);
134     int m = (lx + rx) >> 1;
135     assign(l, r, v, x * 2 + 1, lx, m);
136     assign(l, r, v, x * 2 + 2, m + 1, rx);
137     tree[x] = merge(tree[x * 2 + 1], tree[x * 2 + 2]);
138 }
139
140 void add(int l, int r, ll v, int x, int lx, int rx) {
141     if (lx > r || rx < l) return;
142     if (lx >= l && rx <= r) return apply_add(x, lx, rx, v);
143     push(x, lx, rx);
144     int m = (lx + rx) >> 1;
145     add(l, r, v, x * 2 + 1, lx, m);
146     add(l, r, v, x * 2 + 2, m + 1, rx);
147     tree[x] = merge(tree[x * 2 + 1], tree[x * 2 + 2]);
148 }

```

```

149
150 ll sum(int l, int r, int x, int lx, int rx) {
151     if (lx > r || rx < l) return 0;
152     if (lx >= l && rx <= r) return tree[x].sum;
153     push(x, lx, rx);
154     int m = (lx + rx) >> 1;
155     return sum(l, r, x * 2 + 1, lx, m) + sum(l, r, x * 2 + 2, m + 1, rx);
156 }
157
158 ll qmin(int l, int r, int x, int lx, int rx) {
159     if (lx > r || rx < l) return INF;
160     if (lx >= l && rx <= r) return tree[x].mn1;
161     push(x, lx, rx);
162     int m = (lx + rx) >> 1;
163     return min(qmin(l, r, x * 2 + 1, lx, m), qmin(l, r, x * 2 + 2, m + 1, rx));
164 }
165
166 ll qmax(int l, int r, int x, int lx, int rx) {
167     if (lx > r || rx < l) return -INF;
168     if (lx >= l && rx <= r) return tree[x].mx1;
169     push(x, lx, rx);
170     int m = (lx + rx) >> 1;
171     return max(qmax(l, r, x * 2 + 1, lx, m), qmax(l, r, x * 2 + 2, m + 1, rx));
172 }
173
174 ll qgcd(int l, int r, int x, int lx, int rx) {
175     if (lx > r || rx < l) return 0;
176     if (lx >= l && rx <= r) {
177         ll ans = gcd(tree[x].d_gcd, abs(tree[x].mx1));
178         if (tree[x].mx2 != -INF) ans = gcd(ans, abs(tree[x].mx2 - tree[x].mx1));
179         if (tree[x].mn2 != INF) ans = gcd(ans, abs(tree[x].mn2 - tree[x].mn1));
180         return ans;
181     }
182     push(x, lx, rx);
183     int m = (lx + rx) >> 1;
184     return gcd(qgcd(l, r, x * 2 + 1, lx, m), qgcd(l, r, x * 2 + 2, m + 1, rx));
185 }
186
187 // Public Wrappers
188 void chmin(int l, int r, ll v) { chmin(l, r, v, 0, 0, sz - 1); }
189 void chmax(int l, int r, ll v) { chmax(l, r, v, 0, 0, sz - 1); }
190 void assign(int l, int r, ll v) { assign(l, r, v, 0, 0, sz - 1); }
191 void add(int l, int r, ll v) { add(l, r, v, 0, 0, sz - 1); }
192 ll qsum(int l, int r) { return sum(l, r, 0, 0, sz - 1); }
193 ll qmin(int l, int r) { return qmin(l, r, 0, 0, sz - 1); }
194 ll qmax(int l, int r) { return qmax(l, r, 0, 0, sz - 1); }
195 ll qgcd(int l, int r) { return qgcd(l, r, 0, 0, sz - 1); }
196 };

```

Dynamic Persistent Segment Tree

```

1  template<class info>
2  struct DPSGT {
3      struct node { int l = 0, r = 0; info v; };

```

```

4     vector<node> tr;
5     int n;
6
7     explicit DPSGT(int n = 1e9, int expectedOps = 1) : n(n), tr(1) {
8         tr.reserve(expectedOps * __lg(n) * 2);
9     }
10
11     info get(int x, int lx, int rx, int l, int r) {
12         if (!x || lx > r || l > rx) return info();
13         if (lx >= l && rx <= r) return tr[x].v;
14         int m = (lx + rx) >> 1;
15         return get(tr[x].l, lx, m, l, r) + get(tr[x].r, m + 1, rx, l, r);
16     }
17
18     int set(int x, int lx, int rx, int i, info val) {
19         if (i < lx || i > rx) return x;
20         int nx = tr.size();
21         tr.push_back(tr[x]);
22         if (lx == rx) return tr[nx].v = val, nx;
23         int m = (lx + rx) >> 1;
24         tr[nx].l = set(tr[nx].l, lx, m, i, val);
25         tr[nx].r = set(tr[nx].r, m + 1, rx, i, val);
26         return tr[nx].v = tr[tr[nx].l].v + tr[tr[nx].r].v, nx;
27     }
28
29     int set(int root, int i, info v) { return set(root, 0, n, i, v); }
30     info get(int root, int l, int r) { return get(root, 0, n, l, r); }
31 };

```

Wavelet Tree

```

1 struct WaveletTree {
2     int lo, hi;
3     WaveletTree *lc = 0, *rc = 0;
4     vector<int> b;
5
6     template<class It>
7     WaveletTree(It from, It to, int x, int y) : lo(x), hi(y) {
8         if (from >= to || lo == hi) return;
9         int mid = lo + (hi - lo) / 2;
10
11         b.reserve(distance(from, to) + 1);
12         b.push_back(0);
13         for (auto it = from; it != to; ++it) {
14             b.push_back(b.back() + (*it <= mid));
15         }
16
17         auto pivot = stable_partition(from, to, [mid](int val)
18             { return val <= mid; });
19         lc = new WaveletTree(from, pivot, lo, mid);
20         rc = new WaveletTree(pivot, to, mid + 1, hi);
21     }
22
23     // the passed vector is changed, pass a copy in the main
24     // Takes a vector of integers, and the min/max values in it.

```



```

25 // The vector should be 0-indexed but queries works as 1-based
26 WaveletTree(vector<int>& a, int x, int y) :
27 WaveletTree(a.begin(), a.end(), x, y) {}
28
29 ~WaveletTree() { delete lc; delete rc; }
30
31 int kth(int l, int r, int k) {
32     if (l > r) return 0;
33     if (lo == hi) return lo;
34     int in_left = b[r] - b[l - 1], lb = b[l - 1], rb = b[r];
35     if (k <= in_left) return lc->kth(lb + 1, rb, k);
36     return rc->kth(l - lb, r - rb, k - in_left);
37 }
38
39 int LTE(int l, int r, int k) {
40     if (l > r || k < lo) return 0;
41     if (hi <= k) return r - l + 1;
42     int lb = b[l - 1], rb = b[r];
43     return lc->LTE(lb + 1, rb, k) + rc->LTE(l - lb, r - rb, k);
44 }
45
46 int count(int l, int r, int k) {
47     if (l > r || k < lo || k > hi) return 0;
48     if (lo == hi) return r - l + 1;
49     int mid = lo + (hi - lo) / 2, lb = b[l - 1], rb = b[r];
50     if (k <= mid) return lc->count(lb + 1, rb, k);
51     return rc->count(l - lb, r - rb, k);
52 }
53 };

```

Implicit Treap

```

1 // You must manually track the root of your tree in your main function:
2 //     Treap tree;
3 //     int root = 0;
4 // 1. Insert val at index i:
5 //     int l, r;
6 //     tree.split(root, i, l, r);
7 //     int mid = tree.new_node(val);
8 //     root = tree.merge(tree.merge(l, mid), r);
9 //
10 // 2. Delete the element at index i:
11 //     auto [l, mid, r] = tree.split(root, i, i);
12 //     root = tree.merge(l, r); // mid is safely dropped and ignored
13 //
14 // 3. Range Sum for subarray [L, R]:
15 //     auto [l, mid, r] = tree.split(root, L, R);
16 //     long long ans = tree.tr[mid].sum;
17 //     root = tree.merge(tree.merge(l, mid), r); // ALWAYS merge back!
18 //
19 // 4. Range Reverse for subarray [L, R]:
20 //     auto [l, mid, r] = tree.split(root, L, R);
21 //     tree.tr[mid].rev ^= 1; // apply the lazy tag
22 //     root = tree.merge(tree.merge(l, mid), r); // ALWAYS merge back!
23 // -----

```

```

24
25 mt19937 rnd(time(nullptr));
26 struct Treap {
27     struct node {
28         uint32_t pri = rnd();
29         int sz = 1, l = 0, r = 0, val{};
30         int64_t sum{};
31         bool rev = false;
32         node(int x) : sum(val = x) { }
33     };
34     vector<node> tr;
35
36     // tr[0] acts as a dummy/null node to avoid segmentation faults
37     Treap() : tr(1, 0) { tr[0].sz = 0; }
38
39     inline void pull(int x) {
40         tr[x].sz = tr[tr[x].l].sz + tr[tr[x].r].sz + 1;
41         tr[x].sum = tr[tr[x].l].sum + tr[tr[x].r].sum + tr[x].val;
42     }
43
44     inline void push(int x) {
45         if(tr[x].rev) {
46             tr[tr[x].l].rev ^= 1, tr[tr[x].r].rev ^= 1;
47             swap(tr[x].l, tr[x].r);
48             tr[x].rev = false;
49         }
50     }
51
52     // Returns the index of the newly created node
53     inline int new_node(int val) {
54         tr.emplace_back(val);
55         return int(tr.size()) - 1;
56     }
57
58     // Concatenates treap rx to the right of lx. Returns the new root.
59     int merge(int lx, int rx) {
60         if(!lx || !rx) return rx + lx;
61         push(lx), push(rx);
62         if(tr[lx].pri < tr[rx].pri)
63             return tr[rx].l = merge(lx, tr[rx].l), pull(rx), rx;
64         return tr[lx].r = merge(tr[lx].r, rx), pull(lx), lx;
65     }
66
67     // Splits treap x into lx (first k elements) and rx (the rest).
68     void split(int x, int k, int &lx, int &rx) {
69         if(!x) return lx = rx = 0, void();
70         push(x);
71         if(k <= tr[tr[x].l].sz) split(tr[x].l, k, lx, tr[x].l), pull(rx = x);
72         else split(tr[x].r, k - tr[tr[x].l].sz - 1, tr[x].r, rx), pull(lx = x);
73     }
74
75     // Helper to extract a specific subarray [l, r] into the middle partition
76     array<int, 3> split(int x, int l, int r) {
77         int a, b, c;
78         split(x, r + 1, b, c);
79         split(b, l, a, b);
80         return {a, b, c};

```

```

81     }
82 };

```

Sparse Table

```

1  struct ST {
2      static const int K = 18, N = 2e5 + 5;
3      int st[K][N];
4
5      void build(int n, const vector<int>& a) {
6          for(int i = 0; i < n; i++) st[0][i] = a[i];
7          for(int k = 1; k < K; k++)
8              for(int i = 0; i + (1 << k) <= n; i++)
9                  st[k][i] = min(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
10     }
11
12     int query(int l, int r) {
13         int k = __lg(r - l + 1);
14         return min(st[k][l], st[k][r - (1 << k) + 1]);
15     }
16 };
17
18
19 template<class T, class F>
20 struct sparse {
21     int n, Log;
22     vector<vector<T>> table;
23     F merge;
24     T id;
25
26     sparse(const vector<T>& arr, F merge, T id = T()) :
27         n(arr.size()), Log(__lg(n) + 1), table(Log, vector<T>(n)),
28         merge(merge), id(id) {
29         table[0] = arr;
30         for (int l = 1; l < Log; l++) {
31             for (int i = 0; i + (1 << (l - 1)) < n; i++) {
32                 table[l][i] = merge(table[l - 1][i],
33                                     table[l - 1][i + (1 << (l - 1))]);
34             }
35         }
36     }
37
38     T query(int l, int r) {
39         if (l > r) return id;
40         int len = __lg(r - l + 1);
41         return merge(table[len][l], table[len][r - (1 << len) + 1]);
42     }
43
44     T query_log(int l, int r) {
45         T res = id;
46         bool first = true;
47         for (int j = Log - 1; j >= 0; j--) {
48             if (1 << j <= r - l + 1) {
49                 res = first ? table[j][l] : merge(res, table[j][l]);
50                 first = false;

```

```

51         l += 1 << j;
52     }
53 }
54 return res;
55 }
56 };
57 // sparse st(a, [](int x, int y) { return min(x, y); }, INT_MAX);

```

2D Sparse Table

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  template<class T, class F>
5  struct sparse2d {
6      int n, m, Kx, Ky;
7      vector<int> lgx, lgy;
8      vector<vector<vector<T>>>> st;
9      F merge;
10
11      sparse2d(const vector<vector<T>>& a, F merge) : merge(merge) {
12          n = (int)a.size();
13          m = (int)a[0].size();
14
15          lgx.resize(n + 1);
16          lgy.resize(m + 1);
17          for (int i = 2; i <= n; ++i) lgx[i] = lgx[i / 2] + 1;
18          for (int j = 2; j <= m; ++j) lgy[j] = lgy[j / 2] + 1;
19
20          Kx = lgx[n];
21          Ky = lgy[m];
22
23          st.assign(Kx + 1, vector<vector<vector<T>>>(Ky + 1));
24
25          st[0][0] = a;
26
27          // Build along columns for kx = 0
28          for (int ky = 1; ky <= Ky; ++ky) {
29              int len = 1 << ky;
30              int half = len >> 1;
31              st[0][ky].assign(n, vector<T>(m - len + 1));
32              for (int i = 0; i < n; ++i) {
33                  for (int j = 0; j + len <= m; ++j) {
34                      st[0][ky][i][j] = merge(st[0][ky - 1][i][j],
35                                              st[0][ky - 1][i][j + half]);
36                  }
37              }
38          }
39
40          // Build along rows for all ky
41          for (int kx = 1; kx <= Kx; ++kx) {
42              int lenx = 1 << kx;
43              int halfx = lenx >> 1;
44              for (int ky = 0; ky <= Ky; ++ky) {
45                  int leny = 1 << ky;
46                  st[kx][ky].assign(n - lenx + 1, vector<T>(m - leny + 1));

```

```

47         for (int i = 0; i + lenx <= n; ++i) {
48             for (int j = 0; j + leny <= m; ++j) {
49                 st[kx][ky][i][j] = merge(st[kx - 1][ky][i][j],
50                                           st[kx - 1][ky][i + halfx][j]);
51             }
52         }
53     }
54 }
55
56 // inclusive coordinates: [x1..x2], [y1..y2]
57 T query(int x1, int y1, int x2, int y2) const {
58     int kx = lgx[x2 - x1 + 1];
59     int ky = lgy[y2 - y1 + 1];
60     int dx = 1 << kx;
61     int dy = 1 << ky;
62
63     const T& a = st[kx][ky][x1][y1];
64     const T& b = st[kx][ky][x2 - dx + 1][y1];
65     const T& c = st[kx][ky][x1][y2 - dy + 1];
66     const T& d = st[kx][ky][x2 - dx + 1][y2 - dy + 1];
67
68     return merge(merge(a, b), merge(c, d));
69 }
70 };
71

```

DSU

```

1  struct DSU {
2      vector<int> p, sz;
3      int n, comps;
4      DSU(int _n = 0) { init(_n); }
5      void init(int _n) {
6          n = _n + 10; comps = _n;
7          p.resize(n); sz.assign(n, 1);
8          iota(p.begin(), p.end(), 0);
9      }
10     int find(int u) { return u == p[u] ? u : p[u] = find(p[u]); }
11     bool unite(int u, int v) {
12         u = find(u), v = find(v);
13         if (u == v) return 0;
14         if (sz[u] < sz[v]) swap(u, v);
15         p[v] = u; sz[u] += sz[v]; comps--;
16         return 1;
17     }
18     bool same(int u, int v) { return find(u) == find(v); }
19     int size(int u) { return sz[find(u)]; }
20     int size() { return comps; }
21 };

```

Persistent DSU

```

1  struct persistent_DSU {
2      vector<vector<pair<int, int> > > par;

```

```

3    persistent_DSU(int n) : par(n + 1, {{-1, 0}}) { }
4
5    int time = 0;
6    bool merge(int u, int v) {
7        ++time;
8        if ((u = root(u, time)) == (v = root(v, time)))
9            return 0;
10       if (par[u].back().first > par[v].back().first)
11           swap(u, v);
12       par[u].push_back({par[u].back().first + par[v].back().first, time});
13       par[v].push_back({u, time}); // par[v] = u
14       return 1;
15   }
16
17   bool same(int u, int v, int t) {
18       return root(u, t) == root(v, t);
19   }
20
21   int root(int u, int t) {
22       // root of u at time t
23       if (par[u].back().first >= 0 && par[u].back().second <= t)
24           return root(par[u].back().first, t);
25       return u;
26   }
27
28   int size(int u, int t) { // O(log)
29       // size of the component of u at time t
30       u = root(u, t);
31       int l = 0, r = (int) par[u].size() - 1, ans = 0;
32       while (l <= r) {
33           int mid = l + r >> 1;
34           if (par[u][mid].second <= t)
35               ans = mid, l = mid + 1;
36           else
37               r = mid - 1;
38       }
39       return -par[u][ans].first;
40   }
41   };

```

Bipartite DSU

```

1    // Maintains whether each component is bipartite
2    struct BipartiteDSU {
3        vector<int> sz, bipartite;
4        vector<pair<int, int> > par;
5
6        BipartiteDSU(int n) : par(n), sz(n, 1), bipartite(n) {
7            for (int i = 0; i < n; ++i) {
8                par[i] = {i, 0};
9            }
10       }
11
12       pair<int, int> find(int u) {
13           if (u == par[u].first) return {u, 0};
14           int parity = par[u].second;

```

```

15         par[u] = find(par[u].first);
16         par[u].second ^= parity;
17         return par[u];
18     }
19
20     bool same(int x, int y) {
21         return find(x).first == find(y).first;
22     }
23
24     bool join(int u, int v) {
25         pair<int, int> pu = find(u);
26         pair<int, int> pv = find(v);
27         u = pu.first;
28         v = pv.first;
29         int x = pu.second, y = pv.second;
30         if (u == v) {
31             if (x == y) bipartite[u] = false;
32             return false;
33         }
34         if (sz[u] < sz[v]) swap(u, v);
35         par[v] = {u, x ^ y ^ 1};
36         bipartite[u] &= bipartite[v];
37         sz[u] += sz[v];
38         return true;
39     }
40
41     int size(int x) { return sz[find(x).first]; }
42 };

```

Monotonic Stack / Queue

```

1  template<class T>
2  struct Mono_stack {
3      stack<pair<T, T> > st;
4
5      void push(const T &val) {
6          if (st.empty()) st.emplace(val, val);
7          else st.emplace(val, max(val, st.top().second));
8      }
9
10     void pop() { st.pop(); }
11     bool empty() { return st.empty(); }
12     int size() { return st.size(); }
13     T top() { return st.top().first; }
14     T max() { return st.top().second; }
15 };
16
17 template<class T>
18 struct Mono_queue {
19     Mono_stack<T> pop_st, push_st;
20
21     void push(const T &val) { push_st.push(val); }
22
23     void move() {
24         if (pop_st.size()) return;
25         while (!push_st.empty()) pop_st.push(push_st.top()), push_st.pop();

```

```

26     }
27
28     void pop() { move(); pop_st.pop(); }
29     bool empty() { return pop_st.empty() && push_st.empty(); }
30     int size() { return pop_st.size() + push_st.size(); }
31     T top() { move(); return pop_st.top(); }
32
33     T max() {
34         if (pop_st.empty()) return push_st.max();
35         if (push_st.empty()) return pop_st.max();
36         return max(push_st.max(), pop_st.max());
37     }
38 };

```

Ordered Data Structures (pb_ds)

```

1  #include <ext/pb_ds/assoc_container.hpp>
2  #include <ext/pb_ds/tree_policy.hpp>
3  using namespace __gnu_pbds;
4  template <typename T> using ordered_set = tree<T, null_type, less<T>,
5      rb_tree_tag, tree_order_statistics_node_update>;
6  template <typename T, typename R> using ordered_map = tree<T, R, less<T>,
7      rb_tree_tag, tree_order_statistics_node_update>;

```

BucketList

```

1  template<class T>
2  struct BucketList {
3      int siz = 0;
4      vector<vector<T>> > a;
5      static constexpr int SPLIT_RATIO = 28;
6
7      void insert(int i, T x) {
8          if (siz == 0) {
9              siz = 1;
10             a = {{x}};
11             return;
12         }
13         siz++;
14         for (ll bi = 0; bi < size(a); bi++) {
15             auto &bucket = a[bi];
16             if (i <= size(bucket)) {
17                 bucket.insert(begin(bucket) + i, x);
18                 if (size(bucket) > size(a) * SPLIT_RATIO) {
19                     auto L = end(bucket) - size(bucket) / 2, R = end(bucket);
20                     a.emplace(begin(a) + bi + 1, L, R);
21                     // bucket might be broken
22                     a[bi].erase(L, R);
23                 }
24                 return;
25             }
26             i -= size(bucket);
27         }
28     }

```



```

29
30 void erase(int i) {
31     siz--;
32     for (int bi = 0; bi < size(a); bi++) {
33         auto& bucket = a[bi];
34         if (i < size(bucket)) {
35             bucket.erase(begin(bucket) + i);
36             if (bucket.empty()) a.erase(begin(a) + bi);
37             return;
38         }
39         i -= size(bucket);
40     }
41 }
42
43 T& access(int i) {
44     for (auto& bucket : a) {
45         if (i < size(bucket)) return bucket[i];
46         i -= size(bucket);
47     }
48     assert(false);
49 }
50 };

```

Full Dynamic Array

```

1  mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
2  template<typename T>
3  struct DynArray {
4      struct Node {
5          T val; int pri, sz;
6          Node *l, *r;
7          Node(const T& v) : val(v), pri(rng()), sz(1), l(0), r(0) {}
8      };
9
10     Node* root = 0;
11
12     static int sz(Node* t) { return t ? t->sz : 0; }
13     static void pull(Node* t) { if (t) t->sz = 1 + sz(t->l) + sz(t->r); }
14
15     static void split(Node* t, int k, Node*& l, Node*& r) {
16         if (!t) { l = r = 0; return; }
17         if (sz(t->l) < k) {
18             split(t->r, k - sz(t->l) - 1, t->r, r);
19             l = t;
20         } else {
21             split(t->l, k, l, t->l);
22             r = t;
23         }
24         pull(t);
25     }
26
27     static void merge(Node*& t, Node* l, Node* r) {
28         if (!l || !r) { t = l ? l : r; return; }
29         if (l->pri > r->pri) {
30             merge(l->r, l->r, r);

```

```

31         t = l;
32     } else {
33         merge(r->l, l, r->l);
34         t = r;
35     }
36     pull(t);
37 }
38
39 void insert(int pos, const T& val) {
40     Node *l, *r;
41     split(root, pos, l, r);
42     Node* nd = new Node(val);
43     merge(l, l, nd);
44     merge(root, l, r);
45 }
46
47 void erase(int pos) {
48     Node *l, *m, *r;
49     split(root, pos, l, m);
50     split(m, 1, m, r);
51     delete m;
52     merge(root, l, r);
53 }
54
55 T& access(int pos) {
56     Node* t = root;
57     while (true) {
58         int left = sz(t->l);
59         if (pos < left) t = t->l;
60         else if (pos == left) return t->val;
61         else { pos -= left + 1; t = t->r; }
62     }
63 }
64
65 int size() { return sz(root); }
66 };

```

Count below

```

1 // counts how many pairs that sum <= limit
2 template <typename T>
3 ll count_below(vector<T>& v, int sz, ll Limit){
4     // unique pairs such that v[i]+v[j] <= limit
5     assert(is_sorted(v.begin(), v.end()));
6     ll total = 0;
7     for (int l = 0, r = sz-1; l < r; l++){
8         while(r > l && v[l] + v[r] > Limit)
9             r--;
10        total += max(0, r-l);
11    }
12    return total;
13 }

```